Türkan Begüm Ahmetoğlu
31321

Report of PA2

Thread Safety in the TDQ Implementation:
The TaskDeque struct contains a pthread_mutex_t lock field that is used to protect all access to its internal state. Every operation that reads or modifies the deque such as submitTask, fetchTask, and fetchTaskFromOthers acquires this lock at the start and releases it before returning, including on early-exit paths such as when the queue is found to be empty. Without this, two threads submitting tasks simultaneously could both observe an empty queue and corrupt the head and tail pointers.

Work stealing in fetchTaskFromOthers locks only one foreign queue at a time and releases it immediately after removing a single task. The thread never holds two queue locks simultaneously, which eliminates any risk of deadlock from lock-ordering violations.

A second, independent mutex called in_degree_lock is embedded in each Task struct and protects the in_degree counter during dependency resolution. When a task finishes, its dependents must have their in_degree decremented. Multiple threads can finish tasks that share a dependent at the same time. Without the per-task lock, two threads could both read in_degree == 1, both decrement it, and both submit the same dependent task, causing it to run twice. The in_degree_lock prevents this race by ensuring the decrement and the zero-check are performed atomically together.

Load Balancing and Work Stealing:
Each of the NUM_CORES threads owns a private TaskDeque. At startup, the root tasks are distributed across the queues in a round-robin fashion inside main(), so the initial workload is spread evenly before any thread begins executing.

During execution, each thread runs the processJobs loop. It always tries to fetch a task from its own queue first by calling fetchTask, which removes from the head. If the local queue is empty, the thread does not idle immediately; rather it calls fetchTaskFromOthers, which iterates through all global_queues in order and attempts to steal a task from any other non-empty queue. Stealing is done from the tail of the victim queue, while the owner consumes from the head. This separation reduces contention because the owner and the thief are accessing opposite ends of the deque. If fetchTaskFromOthers also returns nothing, the thread calls usleep(1000) and retries, avoiding a busy spin that would waste CPU cycles.

Additionally, the load balancing scheme is enhanced by using two threshold values, namely a LOW_WATERMARK and a HIGH_WATERMARK, to regulate when a core should steal work or give away work. When the size of a core's local queue drops below the LOW_WATERMARK, the thread attempts to fetch tasks from other cores using fetchTaskFromOthers. This ensures that underutilized cores do not remain idle and can actively participate in processing by borrowing tasks from busier cores.

On the other hand, when the size of a core's queue exceeds the HIGH_WATERMARK, the core proactively shares some of its workload with other cores. This is implemented through the shareTasks function, where the thread scans other queues and transfers one task to a core whose queue size is below the LOW_WATERMARK. This introduces a push-based balancing mechanism in addition to the pull-based work stealing.

The combination of these two thresholds creates a hybrid load balancing strategy. The LOW_WATERMARK ensures that idle or lightly loaded cores actively seek work, while the HIGH_WATERMARK prevents any single core from becoming a bottleneck by redistributing excess tasks. At the same time, task migration is kept minimal by transferring only one task per cycle, which helps preserve cache affinity and reduces the performance penalty associated with cold starts.

When a task finishes, its dependents are submitted directly to the finishing thread's own local queue. This is a deliberate locality decision: the thread that just completed a task is likely to have a warm cache state relevant to that task's dependents, so keeping them local reduces unnecessary migration overhead.

The overall effect is a hybrid load balancing scheme that combines both pull-based and push-based strategies. Idle or underutilized threads actively fetch work from other cores when their queue size falls below the LOW_WATERMARK, while heavily loaded threads proactively share work with others when their queue size exceeds the HIGH_WATERMARK. This ensures that load balancing happens both reactively and proactively, leading to better overall utilization of all cores while still minimizing unnecessary task migration.

Internal Scheduling Policy:

The scheduling policy used within each core is FIFO (First In, First Out). Tasks are always inserted at the tail of the deque in submitTask and removed from the head in fetchTask. This means tasks are executed in the same order they become ready and are submitted to the queue.

This choice was made for two reasons. First, it fits naturally with DAG execution. When a task finishes and its dependents become ready, they are submitted to the local queue in the order they appear in the dependents array. FIFO ensures they are picked up in that same order, which tends to follow the natural depth-first progression of the DAG. Tasks that become ready earlier are also executed earlier, which means downstream dependents are unblocked sooner rather than later, keeping the pipeline moving.

Second, FIFO is simple and predictable. There is no need to store priority values, sort the queue, or reorder nodes at runtime. The head and tail pointers are sufficient to implement it, and insertion and removal are both O(1) operations. This keeps the overhead of the scheduler itself very low, which matters in a multi-threaded setting where the lock on the queue is a shared resource. So, the less time spent inside the critical section, the less contention between threads.

An alternative such as LIFO could have been used, since stealing from the tail already gives LIFO-like behavior to thieves. However, for the owner's own queue, FIFO was preferred because it avoids starvation of tasks that were submitted early and ensures a more uniform progression through the DAG level by level.

Mutex-Free In-Degree Decrements:

If mutexes were not available, the in_degree decrements would need to be made robust using atomic operations. The C standard library provides atomic types and operations through <stdatomic.h>, and the in_degree field in the Task struct could be changed from a plain int to _Atomic int. The decrement would then be performed using atomic_fetch_sub(&dep->in_degree, 1), which returns the value before the subtraction. This entire read-modify-write operation is guaranteed by the hardware to be indivisible. So that no other thread can observe an intermediate state or perform its own decrement between the read and the write.

The zero-check that follows the decrement must also be done carefully. Since atomic_fetch_sub returns the old value, the thread that observes the old value as 1 knows it is the one that brought in_degree to zero, and it alone should submit the dependent task. This avoids the need for a separate lock around the check, because the atomicity of the fetch-and-subtract already serializes all concurrent decrements and makes exactly one thread the "winner" responsible for submission.

The key reason plain “in_degree–” without any synchronization would be unsafe is that it compiles to a load, a modifier, and a store which are three separate instructions. Two threads executing this concurrently can both load the same value, both decrement it, and both store the same result back, effectively losing one decrement. Atomic operations collapse these three steps into one hardware-guaranteed instruction making the outcome correct regardless of how threads are interleaved.